Gradient-Free Orthogonal Feature Organization for Online Task-Free Class-Incremental Learning

Parinita Nema
parinita22@iiserb.ac.in
Vinod K Kurmi
vinodkk@iiserb.ac.in

Department of Data Science and Engineering
Indian Institute of Science Education and Research Bhopal
India

Abstract

This paper studies the challenging problem of online task-free class-incremental learning, a variant of continual learning where the model learns from a one-pass data stream without access to task identities. Existing methods either rely on memory buffers, which raise privacy and storage concerns, or employ continuous gradient-based feature-space organization, which may suffer from severe overfitting and catastrophic forgetting, especially when exposed to a single-pass data stream with unknown task boundaries. To this end, we propose, Gradient-Free Orthogonal Feature Organization with Drift-Guided Refresh, an exemplar-free feature organization framework that performs gradient-free feature adaptation. Our method enforces orthogonality in the high-dimensional projection layer to reduce correlations among feature directions without requiring extensive training. To maintain previous class boundaries without replaying raw samples, we introduce a covariance-aware and dispersion-enhanced prototype mechanism that maximizes inter-class margins. Moreover, to prevent aggressive overwriting caused by continuous updates, our method detects distributional drift and updates the feature organization only when drift is observed. Extensive empirical experiments on four benchmark datasets, including CIFAR-10, CIFAR-100, CUB-200, and CORe50, demonstrate that our approach significantly outperforms existing methods and achieves state-of-the-art performance.

1 Introduction

In recent years, Continual Learning (CL) has garnered significant interest and achieved impressive performance in visual recognition tasks. Traditional CL paradigms [13, 17, 30, 34, 50] divide data streams into discrete subsets, formally known as tasks, where the goal is to incrementally learn from different tasks without forgetting previously acquired knowledge. However, in real-world applications, visual data streams are rarely so explicitly structured. Classes may smoothly mix and shift over time without clear label-changing signals or task-boundary information. Consequently, while most existing CL methods [1, 14, 25, 30] are designed to preserve knowledge from previous tasks, they fundamentally depend on knowing these explicit task boundaries or task identifiers.

To address this limitation, we focus on the Online Task-Free Class-Incremental Learning (OTFCIL) [23, 39, 59, 67] setup, where a model must learn continuously from a seamless,

© 2026. The copyright of this document resides with its authors.
It may be distributed unchanged freely in print or electronic forms.

single-pass data stream without task divisions or task labels. Compared with conventional class-incremental learning (CIL), this setting is more challenging and practical, because each sample is observed by the model only once, and the model must adapt without knowing when the underlying data distribution or class composition changes. Existing OTFCIL methods can be broadly grouped into replay-based, knowledge-distillation-based, and pretrained-model-based approaches. Replay-based methods [8, 23, 39, 56, 57] store or generate old samples to reduce forgetting, while knowledge-distillation-based methods [4, 21, 29, 66] use a previous model as a teacher during online updates, and pretrained-model-based methods [19, 47, 51, 59] leverage prior knowledge from a robust backbone to reduce forgetting. These methods can be broadly classified into three sub-paradigms: (1) fully or partially fine-tuning the network using lightweight adapters [59]; (2) appending and optimizing learnable prompts while keeping the backbone frozen [51]; and (3) using the pretrained model as a fixed feature extractor together with classifiers, such as nearest class mean (NCM), to incrementally accommodate emerging classes [12, 19].

Despite their potential, existing methods still face important limitations that restrict their effectiveness in the OTFCIL setting. First, most replay-based methods depend heavily on memory buffers. However, in real-world applications, strict privacy regulations and limited memory capacity make such sample-storage-based strategies less practical. Second, while approaches utilizing pre-trained models bypass these storage concerns, they still face an important limitation. For example, in a task-free online stream, the model does not know when a new class distribution emerges or when previously learned knowledge should be consolidated. Therefore, continuous gradient updates of the backbone through prompts, adapters, or classifiers may gradually reshape the feature space according to the current stream. Since the model observes each sample only once, continuous updates may make it overfit to the most recently seen data. This can cause old class representations to drift, increase the overlap between old and new classes, and produce unstable decision boundaries. Furthermore, the absence of explicit task identifiers makes it inherently difficult to determine the optimal timing for representation updates. Naive, continuous optimization aggressively overwrites historical knowledge, whereas updating too infrequently prevents the network from successfully identifying novel concepts.

Motivated by these observations, we pose a critical question: ***Instead of repeatedly fine-tuning representations and relearning the feature space through unstable backpropagation, is it possible to maintain a highly separable feature space using an intelligent, gradient-free update mechanism?*** Building upon this premise, our framework avoids traditional gradient-based training. Instead, we perform feature organization through a robust, closed-form solution driven entirely by streaming online statistics.

To this end, we propose a novel ***Gradient-free Orthogonal Feature Organization with Drift-guided Refresh (GOFOR)*** framework. The proposed method uses a frozen pre-trained backbone to extract normalized feature representations. GOFOR avoids trainable adapters and prompts because, in the OTFCIL setting, they are highly prone to causing forgetting or class overlap. Instead, by utilizing a frozen backbone, we map feature representations into a high-dimensional space using an Orthogonal Random Features (ORF) [68] projection. In standard random Gaussian projections or kernel functions, projection vectors randomly overlap and suffer from high variance. This issue becomes more severe in the OTFCIL setting because the model cannot rely on multi-epoch optimization to correct geometric distortions. To address this, we construct the projection layer using orthogonal random bases. This reduces redundancy among projection directions and provides a more stable high-dimensional feature expansion. In addition to the orthogonal projection layer, another key challenge in

boundary-free streams is representing old and new classes without storing raw samples. Typically, models rely on simple empirical class means to remember old classes. However, in an online stream, these normal means fail because deep features are highly correlated and change shapes as the data distribution shifts, leading to severe geometric distortion and class overlap. To this end, our method introduces a Covariance-Aware and Dispersion-Enhanced Prototype mechanism. Instead of using raw averages, we utilize the running shared covariance of the data stream to decorrelate the features by performing an eigendecomposition, transforming the class centers into a clean, uniform space where distances are less affected by high-variance directions. Within this decorrelated space, we apply a prototype dispersion factor that actively pushes different class centers away from one another. These optimized prototypes are then mapped back to the original feature scale via the inverse covariance transformation. This process organizes old and new class centers into a more discriminative and well-separated configuration, allowing the closed-form solution to establish decision boundaries with reduced overlap.

Furthermore, because online task-free streams completely lack explicit task identifiers, a major challenge is determining when to update the representation without destroying previous knowledge. Existing methods typically rely on continuous, repetitive optimization loops. This unguided fine-tuning or continuous updating overwrites old knowledge and causes severe overfitting to the most recent samples. To solve this, GOFOR introduces a concept drift detector based on the Sliced-Wasserstein Distance (SWD) [41]. By continuously tracking momentum-based 1D feature histograms across random projections, our framework actively monitors the data stream and dynamically triggers model updates only when a statistically significant concept drift is detected. In summary, our main contributions are as follows:

- We propose **GOFOR**, a novel exemplar-free feature organization method that enables gradient-free feature adaptation. Since GOFOR does not rely on trainable parameters, it mitigates catastrophic forgetting caused by repeated optimization and provides an efficient and stable feature-space organization.
- We introduce a **drift-guided refresh mechanism** that monitors class-wise distributional changes and updates the prototypes and classifier only when significant drift is detected.
- We propose an **ORF-based high-dimensional projection** to suppress feature variance and reduce redundant projection directions, enabling effective alignment with covariance-aware prototypes.
- Empirical evaluations demonstrate that GOFOR significantly outperforms existing state-of-the-art (SOTA) methods, achieving (at most) absolute accuracy gains of **6.7%** on CIFAR-10, **9.6%** on CIFAR-100, **9.7%** on CORe50, and **14.5%** on CUB-200, respectively.

2 Related Work

2.1 Class Incremental Learning

In CIL, data arrive as a sequence of tasks, and the model is expected to learn new tasks without forgetting previously learned ones. Existing CIL methods are commonly divided into regularization-based, rehearsal-based, and architecture-based approaches. Regularization-based methods constrain parameter updates by adding regularization terms to preserve previous knowledge, such as EWC [25], LwF [30], and their variants [11, 32, 59]. Rehearsal-based methods reduce forgetting by storing old samples and replaying them when new tasks arrive, including iCaRL [42], EEIL [6], DER [63], and related variants [10, 43, 44, 46]. Some methods further use pseudo-samples, such as FearNet [24] and ILUGAN [62], to re-

duce exemplar storage. Architecture-based methods modify the model structure by freezing important parameters or adding new layers and parameters for new tasks. Representative methods include PackNet [34], FACL [38], PNN [44], DyTox [13] and FOSTER [52]. Beyond these categories, recent studies have explored pre-trained-model-based approaches, such as L2P [58], DualPrompt [57] and CODA-P [49], which use lightweight learnable prompts instead of fully updating model parameters. In addition, analytic CIL methods such as KLDA [35], GACL [70], and AnaCIL [36] have recently improved CL by using fixed representations with direct classifier updates. However, these methods are mainly designed for the conventional CIL setting, where task boundaries are known during training.

2.2 Online Class-Incremental Learning

Online-CIL is a more challenging setting, where data arrive as a continuous stream and each sample is usually observed only once. Several methods have been proposed to address this setup, such as [3, 4, 16, 31, 40, 48]. For example, GDumb [40] and MIR [4] store selected samples in memory and replay them during online learning. OnPro [60], CoPE [10], and DSR [22] exploit online prototype-based information to preserve class knowledge without relying only on raw sample storage. Other methods, such as CVT [55], ACAE-REMIND [53], and Si-Blurry [37], further explore pre-trained-model-based strategies to improve representation stability and reduce forgetting in streaming scenarios.

2.3 Online Task-Free Class-Incremental Learning

OTFCIL removes task boundaries during training, and each data sample is visible only once. To address OTFCIL, several methods train models from scratch. For example, CoPE [10] directly learns evolving prototypes from the online stream. Similarly, GMED [23] uses gradient-based criteria to select and store samples from streaming data. However, these methods are trained from scratch, which limits their ability to learn robust representations from a single-pass data stream without task identifiers. To overcome this limitation, some methods use pretrained models. For example, Ensemble [47] employs pretrained backbones and learns only a lightweight classifier. DSDM [39] constructs a memory unit pool to form unit and label vectors. DYSON [19] uses neural collapse to dynamically organize the feature space through prototypes. I^2CANSAY [12] generates pseudo-features from feature distributions and uses them to reduce forgetting. Recent methods have also explored prompt- or adapter-based learning in the OTFCIL setting. For instance, Online-LoRA [59] uses low-rank adapters and fine-tunes them during online learning, while F^2OCL [61] uses prompts to learn from the online stream. However, these methods still depend on gradient-based training for model adaptation. In contrast, our proposed GOFOR performs continual adaptation through closed-form updates without repeatedly optimizing the backbone, prompts, or adapters.

3 Preliminary

3.1 Orthogonal Random Features

Orthogonal Random Features (ORF) approximate shift-invariant kernels by projecting input data into a higher-dimensional space. Given a feature vector $\mathbf{x} \in \mathbb{R}^d$, the projected representation is obtained via a fixed weight matrix $\mathbf{W} \in \mathbb{R}^{D\times d}$ such that $\mathbf{z} = \sigma(\mathbf{W}\mathbf{x}) \in \mathbb{R}^D$, where $\sigma(\cdot)$ is a non-linear activation function. ORF constructs the weight matrix using orthogonal basis vectors. For cases where the projection dimension equals the input dimension ($D = d$),

the matrix satisfies the exact isometry condition: $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_d$, where $\mathbf{I}_d$ is the $d \times d$ identity matrix. In over-parameterized regimes requiring high-dimensional expansion ($D > d$), the full weight matrix is constructed by vertically stacking $k = \lceil D/d \rceil$ independent orthogonal blocks and retaining the first D rows:

$$\mathbf{W} = \begin{bmatrix} \mathbf{Q}_1^\top & \mathbf{Q}_2^\top & \cdots & \mathbf{Q}_k^\top \end{bmatrix}_{1:D}^\top, \qquad \mathbf{W} \in \mathbb{R}^{D \times d}. \tag{1}$$

Each block $\mathbf{Q}_i \in \mathbb{R}^{d \times d}$ is generated by applying a QR decomposition to a random Gaussian matrix $\mathbf{A}_i \in \mathbb{R}^{d \times d}$. By factorizing $\mathbf{A}_i = \mathbf{Q}_i \mathbf{U}_i$, where $\mathbf{Q}_i$ is orthogonal and $\mathbf{U}_i$ is upper triangular, we extract the orthogonal basis. This guarantees every block independently maintains strict orthogonality:

$$\mathbf{Q}_i^\top \mathbf{Q}_i = \mathbf{I}_d, \quad \forall i \in \{1, \ldots, k\}. \tag{2}$$

For $D > d$, this orthogonality condition is imposed independently within each QR-generated block rather than on the full stacked matrix.

3.2 Sliced-Wasserstein Distance

The Wasserstein distance quantifies the structural discrepancy between two probability distributions by calculating the minimum cost required to transport one distribution to another. Let $\mu, \nu \in \mathcal{P}(\mathbb{R}^d)$ be two probability distributions and let $\omega \in \mathbb{S}^{d-1}$ be a unit projection direction. The projected one-dimensional distributions are denoted as μ_ω and ν_ω, where each sample is projected by $x \mapsto \omega^\top x$. For the projected distributions, the one-dimensional p-Wasserstein distance is $W_p(\mu_\omega, \nu_\omega) = \left(\int_0^1 \left| F_{\mu_\omega}^{-1}(s) - F_{\nu_\omega}^{-1}(s) \right|^p ds \right)^{1/p}$, where $F_{\mu_\omega}^{-1}$ and $F_{\nu_\omega}^{-1}$ are inverse cumulative distribution functions. The sliced form averages this distance over all uniform projection directions $\sigma(\omega)$ on the unit sphere $\mathbb{S}^{d-1}$, which in practice is estimated using L random directions:

$$SW_p(\mu, \nu) = \left(\int_{\mathbb{S}^{d-1}} W_p^p(\mu_\omega, \nu_\omega) d\sigma(\omega) \right)^{1/p} \approx \left(\frac{1}{L} \sum_{\ell=1}^{L} W_p^p(\mu_{\omega_\ell}, \nu_{\omega_\ell}) \right)^{1/p} \tag{3}$$

For $p = 1$, the one-dimensional Wasserstein distance simplifies algebraically and can be computed directly from the standard CDFs as $W_1(\mu_\omega, \nu_\omega) = \int_{\mathbb{R}} |F_{\mu_\omega}(t) - F_{\nu_\omega}(t)| dt$. In streaming implementations where empirical distributions are represented by discrete normalized histograms $\mathbf{p}$ and $\mathbf{q}$ with B bins and bin width Δ, this transport cost becomes an efficient sum of cumulative masses:

$$W_1(\mathbf{p}, \mathbf{q}) = \sum_{b=1}^{B} |F_{\mathbf{p}}(b) - F_{\mathbf{q}}(b)| \Delta \tag{4}$$

where $F_{\mathbf{p}}(b)$ and $F_{\mathbf{q}}(b)$ represent the cumulative probability masses of histograms $\mathbf{p}$ and $\mathbf{q}$, respectively, up to bin b.

4 Methodology

4.1 Problem Formulation

OTFCIL involves a sequence of data batches $\mathcal{S} = \{\mathcal{I}^1, \mathcal{I}^2, \ldots, \mathcal{I}^T\}$ arriving as an input data stream, where T denotes the total number of batches, and $\mathcal{I}^t$ represents the t-th batch. Each batch $\mathcal{I}^t$ consists of N_t training samples, defined as $\mathcal{I}^t = \{(x_i^t, y_i^t)\}_{i=1}^{N_t}$, where x_i^t is the i-th input sample and $y_i^t \in \mathcal{Y}^t$ is its corresponding label. During the training step for batch t, only the current batch $\mathcal{I}^t$ is available for training, while samples from previous batches $\{\mathcal{I}^1, \mathcal{I}^2, \ldots, \mathcal{I}^{t-1}\}$ are no longer accessible. Unlike conventional CIL, OTFCIL does not provide explicit task boundaries. Therefore, the learner must update itself from a seamless

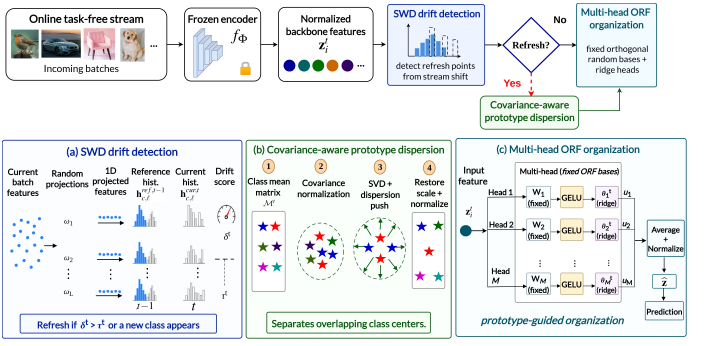


Figure 1: Illustration of the proposed GOFOR framework. GOFOR consists of three main components: (a) SWD drift detection, which monitors distributional changes by comparing current and reference histograms; (b) covariance-aware prototype dispersion, which transforms the class mean matrix $\mathcal{M}^t$ into separated prototype anchors $\mathbf{P}^t$; and (c) multi-head ORF organization, which maps normalized frozen features $\mathbf{z}_i^t$ from Eq. (5) into the organized representation $\widehat{\mathbf{z}}$ for prediction.

stream without knowing when new classes or distributional shifts appear. Following the online learning protocol, each sample is presented only once for training, and the training epoch is fixed to one. At testing time, the model is required to classify samples from all classes encountered up to the current batch, i.e., $\mathcal{Y}^{1:t} = \mathcal{Y}^1 \cup \mathcal{Y}^2 \cup \cdots \cup \mathcal{Y}^t$.

4.2 Framework Overview

The overall framework of our proposed GOFOR is illustrated in Figure 1. Our objective is to organize the feature space in a stable and memory-efficient manner. The framework consists of four major components: (1) a frozen pre-trained feature extractor, (2) a drift-guided refresh mechanism that updates the prototypes and classifier only when necessary. (3) a covariance-aware and dispersion-enhanced online mechanism, and (4) an orthogonal random feature organization module. Let $f_\Phi(\cdot)$ denote a pre-trained visual encoder, where Φ is frozen throughout the stream. Given an input image x_i^t, GOFOR first extracts a normalized feature representation

$$\mathbf{z}_i^t = \frac{f_\Phi(x_i^t)}{\|f_\Phi(x_i^t)\|_2}, \qquad \mathbf{z}_i^t \in \mathbb{R}^d. \tag{5}$$

where d is the feature dimension. In our proposed GOFOR, rather than recomputing the full classification landscape at every step t, the framework computes first- and second-order statistical summaries. The update of the feature organization and the classifier depends on the SWD drift detection module. At time step t, only the current batch $\mathcal{I}^t$ is available, and GOFOR updates compact class-level statistics rather than retraining on raw samples. Therefore, the sample index set for each observed class is $\mathcal{J}_c^t = \{i \mid y_i^t = c\}$, and the mean of class c is computed as $\mathbf{m}_c^t = \frac{1}{|\mathcal{J}_c^t|}\sum_{i\in\mathcal{J}_c^t}\mathbf{z}_i^t$. For each class $c \in \mathcal{Y}^t$, if c is a newly encountered class, i.e., $c \notin \mathcal{Y}^{1:t-1}$, we initialize the global running class mean μ_c^t directly with the current batch vector and if class c already appears in the stream i.e. $c \in \mathcal{Y}^{1:t-1}$, its mean is updated by the count-weighted average using $\mathbf{m}_c^t$:

$$\forall c \in \mathcal{Y}^t, \quad \begin{cases} \mu_c^t = \mathbf{m}_c^t, \quad n_c^t = |\mathcal{J}_c^t|, & \text{if } c \notin \mathcal{Y}^{1:t-1}, \\ \mu_c^t = \frac{n_c^{t-1}\mu_c^{t-1} + |\mathcal{J}_c^t|\mathbf{m}_c^t}{n_c^{t-1}+|\mathcal{J}_c^t|}, \quad n_c^t = n_c^{t-1} + |\mathcal{J}_c^t|, & \text{if } c \in \mathcal{Y}^{1:t-1}. \end{cases} \tag{6}$$

Besides class means, GOFOR computes the covariance matrix to capture global feature dispersion. After updating the class mean, each feature is centered with respect to the corresponding class via $\mathbf{r}_i^t = \mathbf{z}_i^t - \mu_{y_i}^t$. The covariance estimated from the current batch is $\mathbf{C}_b^t = \frac{1}{|\mathcal{I}^t|}\sum_{i=1}^{|\mathcal{I}^t|} \mathbf{r}_i^t (\mathbf{r}_i^t)^\top$. To accumulate the global dispersion, GOFOR first updates the unregularized running covariance Σ^t, and then obtains the regularized shared covariance Σ_ε^t as:

$$\Sigma_\varepsilon^t = \frac{1}{2}\left(\Sigma^t + (\Sigma^t)^\top\right) + \varepsilon \mathbf{I}, \quad \text{where} \quad \Sigma^t = \frac{N_{1:t-1}\Sigma^{t-1} + |\mathcal{I}^t|\mathbf{C}_b^t}{N_{1:t}}. \tag{7}$$

where ε denotes a small positive regularization constant. The resulting compact statistical summary maintained by GOFOR is

$$\mathcal{S}^t = \{(\mu_c^t, n_c^t)\}_{c=1}^{K_t} \cup \{\Sigma_\varepsilon^t\}, \tag{8}$$

where $K_t = |\mathcal{Y}^{1:t}|$ represents the total number of classes encountered so far. The size of $\mathcal{S}^t$ increases with the number of classes rather than the number of samples, and GOFOR does not store raw images or past feature exemplars.

4.3 Sliced-Wasserstein Distance Drift Detection

In OTFCIL, the model does not know when the stream distribution changes. Previous OTFCIL methods update the model after every incoming batch when new samples arrive, which can lead to severe overfitting on novel samples. Repeatedly updating the geometry also increases the computational cost. Therefore, we propose the SWD component, which measures the structural discrepancy between two probability distributions and decides when the model should be updated. Let $\{\omega_\ell\}_{\ell=1}^L$ be a set of random unit projection directions. For a feature $\mathbf{z}$, its projection along direction ℓ is $s_\ell(\mathbf{z}) = \omega_\ell^\top \mathbf{z}$, $\omega_\ell \in \mathbb{S}^{d-1}$. For each class $c \in \mathcal{Y}^t$ appearing in the current batch and each projection direction ω_ℓ, GOFOR builds a normalized current histogram $\mathbf{h}_{c,\ell}^{cur,t} = \mathrm{Hist}_{B,\Delta}\left(\{s_{i,\ell}^t \mid y_i^t = c\}\right)$, where $\mathrm{Hist}_{B,\Delta}(\cdot)$ denotes a normalized histogram operator with B bins and bin width Δ, such that the total probability mass sums to one. In parallel, GOFOR maintains a reference histogram $\mathbf{h}_{c,\ell}^{ref,t-1}$, which summarizes the previous projected distribution of class c along direction ω_ℓ. To evaluate the structural divergence between these two configurations, we substitute the current and reference states directly into the discrete W_1 formulation established in Sec. 3.2, setting $\mathbf{p} = \mathbf{h}_{c,\ell}^{cur,t}$ and $\mathbf{q} = \mathbf{h}_{c,\ell}^{ref,t-1}$. The one-dimensional Wasserstein transport cost is thus calculated as a highly efficient sum of cumulative masses:

$$W_{c,\ell}^t = \sum_{b=1}^{B} \left| F_{\mathbf{h}_{c,\ell}^{cur,t}}(b) - F_{\mathbf{h}_{c,\ell}^{ref,t-1}}(b) \right| \Delta, \tag{9}$$

where $F(b)$ represents the cumulative probability mass of the designated histogram up to bin b. For each class c, the drift score is obtained by averaging the transport cost across all projection directions. Since a refresh decision must be made at the batch level, we use the largest class-wise drift among the classes appearing in the current batch:

$$\delta_c^t = \frac{1}{L}\sum_{\ell=1}^{L} W_{c,\ell}^t, \qquad \delta^t = \max_{c \in \mathcal{Y}^t} \delta_c^t. \tag{10}$$

Here, δ_c^t measures how much the current distribution of class c deviates from its reference distribution, while δ^t is the final drift score used for the update decision. For a newly observed class, the reference histogram is initialized from the current histogram, and the arrival

of the new class activates the update criterion. After computing δ^t, GOFOR maintains it in a recent drift queue $\mathcal{H}_\delta = \{\delta^{t-m}, \ldots, \delta^{t-1}\}$, where m is the history length. The adaptive refresh threshold is defined as $\tau^t = \text{mean}(\mathcal{H}_\delta) + \kappa\,\text{std}(\mathcal{H}_\delta)$, Refresh if $\delta^t > \tau^t$. This adaptive rule prevents GOFOR from refreshing under minor stream fluctuations, while still allowing updates when the current batch shows a statistically larger distributional change than recent drift statistics. Finally, the reference histogram is updated by momentum:

$$\mathbf{h}_{c,\ell}^{ref,t} = \rho \mathbf{h}_{c,\ell}^{ref,t-1} + (1-\rho)\mathbf{h}_{c,\ell}^{cur,t}. \tag{11}$$

SWD is not used as an optimization loss and does not directly adapt the feature distribution. It only provides a task-free signal for deciding when the feature organization and classifier should be refreshed.

Covariance-aware prototype dispersion. When the refresh condition is satisfied, GOFOR reconstructs the class prototypes from the updated online statistical summaries. Existing prototype classifiers directly use class means as prototypes. However, in OTFCIL, old and new classes appear together without clear boundaries, and their means can move close to each other, especially when feature dimensions are highly correlated. Upon receiving a refresh signal, the framework organizes its class means into a matrix $\mathcal{M}^t = [\mu_1^t, \mu_2^t, \ldots, \mu_{K_t}^t]^\top \in \mathbb{R}^{K_t \times d}$. The global center vector of all observed visual categories is evaluated as: $\bar{\mu}^t = \frac{1}{K_t}\sum_{c=1}^{K_t} \mu_c^t$, where $K_t = |\mathcal{Y}^{1:t}|$ denotes the total number of unique classes encountered up to the current stream step. To isolate class locations from directional feature correlation, the centers are projected into a variance-normalized representation space using the regularized global covariance summary: $\widetilde{\mathcal{M}}^t = \left(\mathcal{M}^t - \mathbf{1}_{K_t}(\bar{\mu}^t)^\top\right)(\Sigma_\varepsilon^t)^{-\frac{1}{2}}$, where $\mathbf{1}_{K_t} \in \mathbb{R}^{K_t}$ is an all-one column vector. This projection maps the class means into a covariance-normalized space, where separation is not dominated by high-variance or correlated feature dimensions. To prevent the classes from overlapping, we apply Singular Value Decomposition (SVD) to split this normalized mean matrix into individual components and shift its singular values by a constant factor α: $\widetilde{\mathbf{P}}^t = \mathbf{U}^t(\mathbf{S}^t + \alpha\mathbf{I})(\mathbf{V}^t)^\top$, $\widetilde{\mathcal{M}}^t = \mathbf{U}^t\mathbf{S}^t(\mathbf{V}^t)^\top$, where $\mathbf{U}^t$ and $\mathbf{V}^t$ are orthogonal matrices, and $\mathbf{S}^t$ contains the singular values. The constant $\alpha \geq 0$ acts as a push factor that physically separates the class prototypes from one another. To map these optimized structures back into the valid tracking scale, the vectors are scaled back to the original feature size and localized around the global stream center:

$$\mathbf{P}^t = \text{Norm}\left(\widetilde{\mathbf{P}}^t(\Sigma_\varepsilon^t)^{\frac{1}{2}} + \mathbf{1}_{K_t}(\bar{\mu}^t)^\top\right), \tag{12}$$

where $\text{Norm}(\cdot)$ denotes row-wise ℓ_2-normalization. After normalization, each row of $\mathbf{P}^t = [\mathbf{p}_1^t, \mathbf{p}_2^t, \ldots, \mathbf{p}_{K_t}^t]^\top$ becomes a unit-norm class prototype on the hypersphere $\mathbb{S}^{d-1}$.

These prototypes provide more discriminative class anchors than empirical means, and GOFOR uses them as target coordinates for the subsequent random feature mapping blocks rather than raw class centers. This yields clearer separation between old and new classes while relying only on compact statistical summaries, without retaining historical exemplars.

4.4 Orthogonal Random Feature Projection

After constructing the covariance-aware prototype set, GOFOR needs to map incoming frozen features to the high-dimensional feature space. A direct linear mapping from the original feature space may be insufficient because the frozen backbone features are not explicitly optimized for the current online class sequence. GOFOR therefore expands the features into a high-dimensional nonlinear space using ORF, and then solves the alignment to

the prototype space by closed-form ridge regression. For the m-th head, GOFOR constructs a fixed orthogonal projection matrix $\mathbf{W}_m \in \mathbb{R}^{D\times d}$, where d is the backbone feature dimension and D is the projected dimension. Following the ORF construction in Sec. 3.1, $\mathbf{W}_m$ is obtained from orthogonal bases generated by QR decomposition of random Gaussian matrices $\mathbf{A}_m = \mathbf{Q}_m\mathbf{R}_m, \quad \mathbf{Q}_m^\top\mathbf{Q}_m = \mathbf{I}$. If $D > d$, multiple independent orthogonal blocks are concatenated. Given a normalized feature $\mathbf{z}$, the projected representation is

$$\phi_m(\mathbf{z}) = \sigma(\mathbf{W}_m\mathbf{z}), \qquad \phi_m(\mathbf{z}) \in \mathbb{R}^D, \tag{13}$$

where $\sigma(\cdot)$ denotes the GELU activation. This produces a high-dimensional nonlinear representation while avoiding trainable projection parameters. Each ORF head learns a mapping from the projected representation to the covariance-aware prototype space. Rather than using backpropagation, GOFOR forms two sufficient statistics: the Gram matrix of projected features and the cross matrix between projected class statistics and prototype anchors:

$$\mathbf{G}_m^t = \sum_{\tau\le t}\sum_{i=1}^{N_\tau} \phi_m(\mathbf{z}_i^\tau)\phi_m(\mathbf{z}_i^\tau)^\top, \qquad \mathbf{H}_m^t = \sum_{c=1}^{K_t} n_c^t \eta_{m,c}^t (\mathbf{p}_c^t)^\top, \tag{14}$$

where $\eta_{m,c}^t$ denotes the running mean of ORF-projected features for class c in the m-th head. Using these statistics, the mapping matrix is obtained by ridge regression [45]:

$$\Theta_m^t = \left(\mathbf{G}_m^t + \lambda\mathbf{I}\right)^{-1}\mathbf{H}_m^t, \tag{15}$$

where λ is the ridge regularization parameter. Thus, the feature-to-prototype organization is computed directly from accumulated statistics, without iterative gradient optimization. For an input feature $\mathbf{z}$, the organized output of the m-th head is $\mathbf{u}_m = (\Theta_m^t)^\top\phi_m(\mathbf{z}) \in \mathbb{R}^d$. GOFOR uses M independent ORF heads and combines their outputs through averaging followed by row-wise normalization:

$$\widehat{\mathbf{z}} = \mathrm{Norm}\left(\frac{1}{M}\sum_{m=1}^{M}\mathbf{u}_m\right). \tag{16}$$

The resulting representation $\widehat{\mathbf{z}}$ is used by the final classifier. Through this multi-head ORF organization, GOFOR maps frozen features into a more separable prototype-aligned representation while keeping the entire update process gradient-free.

4.5 Feature-Level Synthesis and Classifier

After updating the ORF mappings, GOFOR builds the final decision branch using statistics-guided feature synthesis. Since previous images are not accessible in OTFCIL, the classifier cannot be fitted using stored raw samples. Instead, GOFOR reuses the compact statistical summary in Eq. (8). For each observed class c, pseudo-features are generated from a Gaussian distribution parameterized by the running class mean and the shared covariance:

$$\widetilde{\mathbf{z}}_{c,j} = \mathrm{Norm}\left(\mu_c^t + \varepsilon_{c,j}\right), \qquad \varepsilon_{c,j} \sim \mathcal{N}\left(\mathbf{0}, \Sigma_\varepsilon^t\right), \quad j = 1,\dots,R, \tag{17}$$

where R denotes the number of pseudo-features generated for each class. These pseudo-features approximate old and newly observed class distributions in the frozen backbone space. Each generated pseudo-feature is then passed through the multi-head ORF organization module defined in Eq. (16). This gives the organized pseudo-feature representation: $\widehat{\widetilde{\mathbf{z}}}_{c,j} = \mathcal{A}^t\left(\widetilde{\mathbf{z}}_{c,j}\right)$, where $\mathcal{A}^t(\cdot)$ denotes the current multi-head ORF organization function. To improve the expressiveness of the final classifier, GOFOR further applies a fixed ORF classifier projection: $\mathbf{q}_{c,j} = \sigma\left(\left(\widehat{\widetilde{\mathbf{z}}}_{c,j}\right)^\top\mathbf{W}_{cls}\right) \in \mathbb{R}^{r_c}$, where $\mathbf{W}_{cls} \in \mathbb{R}^{d\times r_c}$ is a fixed orthogonal random projection matrix for the classifier and r_c is its projected dimension. Let $\mathbf{Q}^t \in \mathbb{R}^{K_tR\times r_c}$

Method	Backbone	Buffer Size	CIFAR-10			Buffer Size	CIFAR-100		
			1-step	2-step	Gaussian		1-step	5-step	Gaussian
ODDL [31]	ResNet18	2k	–	52.7	–	2k	–	27.2	–
SDP [42]		500	–	66.2	76.3	2k	–	–	–
DYSON [19]		0	73.4	74.4	76.5	0	49.6	45.3	47.0
GOFOR (Ours)		0	**86.4**	**86.7**	**86.6**	0	**62.8**	**62.7**	**62.3**
CoPE [10]	ResNet50	1k	–	48.9	–	5k	–	21.6	–
CN-DPM [28]		1k	–	45.2	–	1k	–	20.1	–
GMED [5]		5k	–	47.7	–	5k	–	19.6	–
FCA [36]		0	77.5	78.7	76.1	0	53.3	48.7	53.4
Ensemble [10]		2k	78.3	79.0	50.1	6k	54.1	55.3	39.0
DSDM [33]		2k	79.4	79.6	78.7	6k	54.9	55.3	55.5
DYSON [19]		0	79.5	80.7	79.1	0	58.9	59.2	58.6
I^{2}CANSAY [12]		0	82.7	83.5	83.0	0	59.4	60.7	61.4
GOFOR (Ours)		0	**89.4**	**89.3**	**89.2**	0	**69.0**	**68.7**	**68.1**
L2P [38]	ViT-S/8 dino-ViT8	1k	46.8	61.4	57.5	3k	8.4	27.3	47.8
DualPrompt [37]		1k	49.3	63.7	59.5	3k	10.2	30.5	49.1
CODA [35]		1k	52.7	67.3	65.2	3k	14.3	33.3	56.7
DSDM [33]		1k	85.5	85.6	84.9	3k	61.1	60.8	61.4
DYSON [19]		0	92.6	93.5	93.8	0	77.7	75.6	76.4
I^{2}CANSAY [12]		0	94.4	93.7	93.9	0	78.2	79.2	78.3
GOFOR (Ours)		0	**97.4**	**97.5**	**97.3**	0	**85.7**	**85.8**	**85.1**

Table 1: Performance comparison in the OTFCIL setting in terms of last accuracy (%) on CIFAR-10 and CIFAR-100. Baseline results are taken from DYSON [19] and I^{2}CANSAY [12].

be the matrix obtained by stacking all classifier features $\mathbf{q}_{c,j}$, and let $\mathbf{Y}^t \in \mathbb{R}^{K_t R \times K_t}$ be the corresponding one-hot label matrix. The final classifier is obtained by ridge regression:

$$\Omega^t = \left((\mathbf{Q}^t)^\top \mathbf{Q}^t + \lambda_c \mathbf{I}\right)^{-1} (\mathbf{Q}^t)^\top \mathbf{Y}^t, \tag{18}$$

where λ_c is the classifier regularization coefficient. This step directly computes the classifier weights from generated feature representations and avoids iterative classifier training. During inference, an input image x is first converted into a normalized frozen feature $\mathbf{z}$ using Eq. (5). The feature is then organized by the current ORF module: $\widehat{\mathbf{z}} = \mathcal{A}^t(\mathbf{z})$. The classifier branch computes

$$\mathbf{q} = \sigma\left(\widehat{\mathbf{z}}^\top \mathbf{W}_{cls}\right), \qquad \hat{y} = \arg\max_{c \in \mathcal{Y}^{1:t}} [\mathbf{q}\Omega^t]_c. \tag{19}$$

The ridge classifier in Eq. (18) is used as the main prediction branch, while the prototype branch provides a direct measure of how well the organized representation aligns with the refreshed prototype anchors.

5 Results and Experiments

5.1 Dataset and Evaluation Details

We conduct experiments on CIFAR-10 [27] and CIFAR-100 [27] under the OTFCIL setting to evaluate the proposed GOFOR method. In addition, we use CORe50 [33] for the online-CIL and CUB-200 [61] for the offline task-free incremental learning setting to further validate the generality of our approach.

Datasets. CIFAR-10 [27] is a standard CL benchmark with 60,000, 32×32 color images, including 50,000 training and 10,000 test images across 10 classes. CIFAR-100 [27] has the same resolution and size but consists of 100 fine-grained classes. CUB-200 [61] is a fine-grained bird classification dataset with 200 species, where each class has approximately 30 training and 29 testing images. CORe50 [33] contains 50 object categories captured under diverse visual conditions, with 2,400 training and 900 testing images in our protocol. Beyond the four benchmarks above, we additionally include Split ImageNet-R as a robustness check

under domain shift. ImageNet-R [20] provides renditions (art, cartoons, sketches) of 200 ImageNet classes, and we follow the split construction of DualPrompt [57].

Evaluation metrics. We use two standard CL metrics: Last Accuracy (Last) and Average Accuracy (Avg). Last Accuracy evaluates performance on all encountered classes after the full training stream, while Average Accuracy measures the mean performance across learning steps to reflect stability during incremental learning.

Implementation details. We evaluate GOFOR using three pre-trained backbones: ResNet-18 [18], ResNet-50 [18], and ViT-S/8 [5]. For the OTFCIL setting, we follow the same Gaussian class schedule protocol used in [12, 19]. All baseline numbers in Tables 1, 2, and 3 are taken directly from DYSON [19] and I^{2}CANSAY [12] under identical protocols; we do not re-implement them. The batch size is fixed to 50 for all experiments. For the ORF module, we use $M = 3$ projection heads, and the projected feature dimension is set to $D = 5000$. The ridge regularization parameter is fixed to $\lambda = 10^2$. During classifier refresh, we generate $R = 100$ pseudo features per observed class from the Gaussian distribution. For SWD-based drift detection, we use $L = 32$ random projection directions and construct histograms with 32 bins. The reference histograms are updated with momentum $\rho = 0.9$, and the adaptive threshold multiplier is set to $\kappa = 2.0$.

5.2 Comparison with State-of-The-Arts

Online Task-Free Class-Incremental Learning. We compare GOFOR with recent SOTA approaches, including CoPE [10], CN-DPM [28], GMED [23], FCA [64], Ensemble [47], DSDM [39], DYSON [19], and I^{2}CANSAY [12], under the 1-step, 2-step, and Gaussian schedules. In the n-step protocol, each stage introduces n new classes. As shown in Table 1, GOFOR consistently achieves superior performance across CIFAR-10 and CIFAR-100 using multiple backbones. With the ViT-S/8 backbone, GOFOR improves over I^{2}CANSAY by **3.0**%, **3.8**%, and **3.4**% on CIFAR-10, and by **7.5**%, **6.6**%, and **6.8**% on CIFAR-100 under the 1-step, 2-step, and Gaussian protocols, respectively. A similar trend holds with ResNet-50, where GOFOR yields last-accuracy gains of **6.7**%, **5.8**%, and **6.2**% on CIFAR-10, and **9.6**%, **8.0**%, and **6.7**% on CIFAR-100 across the respective stream configurations. These margins demonstrate that GOFOR consistently outperforms existing baselines on both benchmarks. Crucially, GOFOR secures these results completely without relying on iterative gradient-based training or utilizing a rehearsal memory buffer to store past exemplars.

Online Class-Incremental Learning. We compare our method in the online-CIL setting with recent SOTA methods, including A-GEM [7], MIR [2], GDUMB [40], CV [16], GSS [3], ASER [48], OBC [9], GSA [15], PCR [31], DSDM [39], DYSON [19], and I^2-CANSAY [12]. We conduct experiments on CIFAR-10 and CORe50 and follow the same evaluation protocol as DYSON. For CIFAR-10, the learning process is divided into five stages with a 2-step evaluation protocol. For CORe50, we use 9 stages, where the first stage contains 10 classes and each remaining stage introduces 5 classes. For a fair comparison, we follow the same protocol as prior work [19] and use a pretrained ResNet-18 backbone. The results in Table 2 show that, on CIFAR-10, our method outperforms all compared methods across different memory buffer sizes, including 1K, 2K, and 5K. Specifically, GOFOR improves over DSDM [39] with a 5K buffer and I^{2}CANSAY [12] with no buffer by **5.0**% and **7.3**% in average accuracy and last accuracy, respectively. Similarly, on CORe50, GOFOR achieves improvements of **7.0**% in average accuracy and **9.7**% in last accuracy. These results show that previous methods often rely on repeated model updates and large memory storage, whereas GOFOR uses a drift-guided refresh mechanism to update only when necessary.

Method	CIFAR-10						CORe50					
	1k		2k		5k		1k		2k		5k	
	Avg	Last	Avg	Last	Avg	Last	Avg	Last	Avg	Last	Avg	Last
Buffer size: 1k/2k/5k												
A-GEM [8]	43.0	17.5	59.1	38.3	74.0	59.0	20.7	8.4	21.9	10.3	22.9	11.5
ASER [46]	63.4	46.4	78.2	59.3	83.3	73.1	24.3	12.2	30.8	17.4	32.5	18.5
MIR [4]	67.3	52.2	80.2	66.2	83.4	74.8	33.9	21.1	37.1	24.5	38.1	27.7
GSS [3]	70.3	56.7	73.6	56.3	79.3	64.4	27.8	17.8	31.0	18.9	31.8	21.1
GDUMB [44]	73.8	57.7	83.8	72.4	85.3	75.9	41.2	23.6	48.4	32.7	54.3	41.6
GSA [5]	75.5	63.1	78.8	69.7	79.3	70.2	67.1	53.9	72.9	64.2	73.6	67.4
CV [43]	76.0	62.9	84.9	74.1	86.1	77.0	45.1	26.5	50.7	34.5	56.3	43.1
PCR [58]	76.7	63.5	78.0	65.4	78.9	66.1	74.3	66.3	75.4	66.9	74.4	67.8
OBC [1]	78.1	65.8	84.3	72.8	85.8	76.2	51.4	43.9	56.9	47.2	65.3	54.9
DSDM [45]	80.2	67.0	83.8	72.5	85.6	76.0	43.9	43.3	53.2	50.8	66.3	57.1
Buffer size: 0 (memory-free)												
	Avg		Last		Avg		Last					
DYSON [33]	81.8		74.4		70.7		61.3					
I^2CANSAY [31]	85.3		79.6		77.2		68.8					
GOFOR (Ours)	**90.6**		**86.9**		**84.2**		**78.5**					

Table 2: Online CIL performance comparison on the CIFAR-10 and CORe50 datasets.

Method	CIFAR-100		CUB-200		Mem.
	5-step	2-step	5-step	2-step	
	Avg Acc.		Avg Acc.		
Buffer size: 3k					
EWC [25]	17.9	15.4	18.2	15.9	3k
LwF [30]	37.9	32.8	36.1	29.8	
iCaRL [42]	57.2	52.8	49.6	44.3	
EEIL [6]	49.7	50.3	49.9	44.5	
FearNet [24]	62.5	56.9	52.7	47.8	
ILUGAN [62]	63.1	58.0	54.9	49.7	
DSDM [45]	63.2	63.3	55.5	55.2	
DRC [8]	63.0	61.7	55.1	50.6	
SOUL [54]	64.5	63.8	56.2	53.3	
Buffer size: 0 (memory-free)					
DYSON [33]	71.9	71.7	59.9	59.1	0
I^2CANSAY [31]	69.4	68.2	64.0	63.5	0
GOFOR (Ours)	**79.3**	**79.7**	**77.4**	**78.0**	0

Table 3: Offline task-free CIL performance (average accuracy %) on CIFAR-100 and CUB-200.

Thus, GOFOR surpasses SOTA methods without gradient-based adaptation.

Offline task-free class-incremental learning. We also evaluate the performance of our method under the offline task-free incremental learning setting. We compare GOFOR with SOTA offline continual learning methods, including iCaRL [42], EEIL [6], EWC [25], FearNet [24], LwF [30], ILUGAN [62], DRC [8], and SOUL [54]. For this setting, we also follow the same evaluation protocol as DYSON. As shown in Table 3, our method outperforms recent SOTA approaches. Specifically, on CIFAR-100, GOFOR achieves **7.4**% and **8.0**% higher accuracy under the 5-step and 2-step protocols, respectively. Similarly, on CUB-200, GOFOR obtains **13.4**% and **14.5**% higher accuracy compared with the best existing method under the 5-step and 2-step protocols, respectively. These results show that the proposed GOFOR steadily and significantly outperforms existing baselines by a large margin.

Robustness under domain shift. Beyond the four main benchmarks, we further evaluate GOFOR on the more challenging Split ImageNet-R under the Gaussian OTFCIL protocol using the ViT-S/8 [5] backbone and the same fixed hyperparameters as in all other experiments. As shown in Figure 2, GOFOR outperforms DYSON [19] by a clear margin in both Last and Avg. accuracy, demonstrating that the gains remain robust under substantial domain shift. For reference, the figure also reports the joint-training upper bound obtained by fine-tuning on all data without the single-pass constraint.

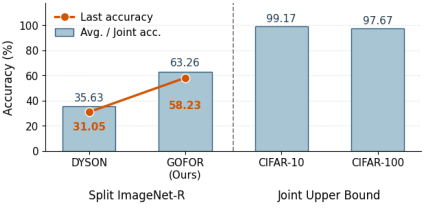


Figure 2: Robustness on Split ImageNet-R (left) and joint-training upper bounds on CIFAR-10/100 (right).

5.3 Ablation Study

Impact of Framework Components. We evaluate the components of GOFOR on CIFAR-100 under the Gaussian protocol in Table 4. Using only the frozen backbone yields the weakest performance, showing that fixed pre-trained features alone are insufficient for OTFCIL. Introducing SWD and covariance-aware dispersion without ORF brings only a modest improvement, suggesting that prototype reorganization is helpful but limited when the model

CIFAR-100 (Gaussian)				Last Acc. (%)
Frozen Backbone	ORF	SWD	Cov.-aware Dispersion	
✓				76.2
✓		✓	✓	78.3
✓	✓	✓		82.5
✓	✓		✓	83.9
✓	✓	✓	✓	**85.1**

Table 4: Component-wise ablation on CIFAR-100 under the Gaussian protocol.

Dataset	Backbone	1-step	2-step	Gaussian
CIFAR-10	ResNet50	89.4	89.3	89.2
	DINO-ResNet50	**92.6**	**92.7**	**92.3**
CIFAR-100	ResNet50	69.0	68.7	68.1
	DINO-ResNet50	**74.51**	**74.51**	**73.58**

Table 5: Last accuracy comparison of GOFOR using supervised and self-supervised frozen ResNet-50 backbones on CIFAR-10 and CIFAR-100 datasets.

CIFAR-100 (Gaussian)				
Projection	PCA	Learned Linear Projection	Gaussian Random Projection	ORF (Ours)
Last Acc. (%)	61.5	62.0	82.6	**85.1**

Table 6: Analysis of different feature projection strategies with ORF on the CIFAR-100 dataset.

CIFAR-100 (Gaussian)			
Refresh Strategy	New Classes	Every N Batches	SWD-guided
Last Acc. (%)	80.2	82.6	**85.1**

Table 7: Comparison of different update mechanisms against the proposed SWD-based strategy on the CIFAR-100 dataset.

still operates directly in the original backbone feature space. The inclusion of ORF leads to a clear performance gain, demonstrating the importance of orthogonal high-dimensional feature expansion. In addition, removing either covariance-aware dispersion or SWD from the full model results in lower accuracy, showing that both discriminative prototype organization and drift-guided refresh are necessary. The complete GOFOR achieves the best last accuracy of 85.1%, confirming that these components are complementary and jointly contribute to stable feature-space organization.

Influence of the Frozen Pre-trained Backbone. To further examine whether the proposed method depends on a specific frozen feature extractor, we evaluate GOFOR with different supervised and self-supervised backbones. Specifically, we compare supervised ResNet-50 and self-supervised DINO-ResNet-50 [5] backbones. As shown in Table 5, GOFOR remains effective across both supervised and self-supervised frozen feature extractors. This demonstrates that the proposed framework is not tied to a particular pre-training strategy and can generalize across different types of visual representations.

Effectiveness of the ORF Projection. To validate the necessity of the proposed ORF module in OTFCIL, we compare it against Principal Component Analysis (PCA), a learned linear projection, and standard Gaussian random projection. As shown in Table 6, PCA and the learned linear layer perform substantially worse. Gaussian random projection improves performance but introduces directional overlap and redundancy in the expanded space. ORF instead achieves the best performance, showing that explicit orthogonality provides more stable and less redundant feature directions for OTFCIL.

Impact of the SWD Mechanism. We further analyze the contribution of the proposed SWD-guided refresh strategy by comparing it with alternative refresh policies, as reported in Table 7. Refreshing only when a new class appears may fail to capture distributional changes within already seen classes, while fixed-interval refresh can update too frequently. In contrast, the proposed SWD-guided strategy refreshes the model only when a clear feature-distribution shift is detected, providing a more adaptive balance between stability and plasticity and yielding the best performance.

Sensitivity Study of the SWD Hyperparameters (L, ρ, κ). We further conduct a sensitivity analysis of the SWD-based drift detection module by varying the number of SWD projections, the momentum value, and the threshold coefficient. As shown in Figure 3, the final accuracy remains stable across different hyperparameter settings. Changing the number of

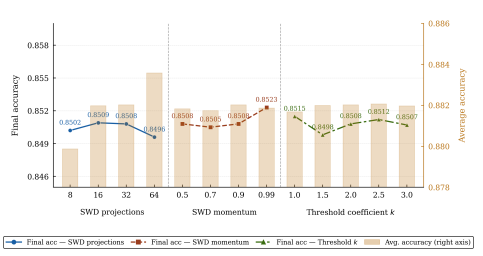


Figure 3: Sensitivity analysis of the SWD-guided refresh mechanism on CIFAR-100 across projection directions L (left), momentum ρ (center), and threshold κ (right).

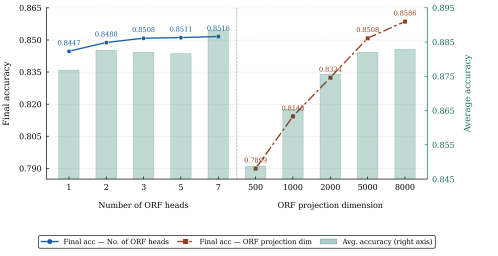


Figure 4: Sensitivity analysis of the ORF module on CIFAR-100 across varying numbers of parallel projection heads M (left) and high-dimensional projection dimensions D (right).

SWD projections and momentum values produces only minor performance variations, indicating that the proposed SWD-guided refresh mechanism is not highly sensitive to specific hyperparameter choices and can provide consistent performance under different configurations. For the threshold coefficient, the accuracy remains stable across different threshold values, indicating that the model is not highly sensitive to the exact refresh frequency.

Analysis of ORF Hyperparameters (M, D)**.** We analyze the effect of ORF heads and projection dimension in Figure 4. Using a single head gives weaker performance, suggesting that one projection is not sufficient to capture stable feature organization in the online stream. Similarly, increasing the ORF dimension improves accuracy by enriching the projected feature space, although the improvement gradually saturates. These results show that the method benefits from multiple ORF heads and a sufficiently large projection dimension without being overly sensitive to the exact setting.

6 Conclusion

In this work, we propose a novel GOFOR framework which contains gradient-free modules specifically designed to tackle the challenges of Online Task-Free Class-Incremental Learning (OTFCIL). We introduce ORF projection, which makes feature representations more separable by projecting them into a high-dimensional orthogonal space. Furthermore, to prevent catastrophic forgetting while maintaining plasticity, we utilize a drift-guided refresh mechanism via SWD. Complementing this, our training-free dispersed prototypes establish discriminative class centers. We perform comprehensive experiments across four benchmark datasets, achieving state-of-the-art performance. These results indicate that gradient-free feature organization coupled with drift-aware refreshing represents a promising direction for stable and efficient continual learning in task-free data streams.

Despite these results, GOFOR has some limitations. Since GOFOR operates on a frozen backbone, its accuracy is bounded by the encoder's representational quality, and this dependency grows under large domain shift. It also relies on class-wise means and a shared covariance that require a sufficient number of samples per class, so extremely low-sample or heavily class-imbalanced streams may weaken these estimates. Finally, the pseudo-features are drawn from a single Gaussian, which may not capture multimodal or long-tailed class distributions; exploring richer generators while preserving the closed-form, gradient-free nature of GOFOR is left to future work.

References

[1] Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *Proceedings of the European conference on computer vision (ECCV)*, pages 139–154, 2018.

[2] Rahaf Aljundi, Eugene Belilovsky, Tinne Tuytelaars, Laurent Charlin, Massimo Caccia, Min Lin, and Lucas Page-Caccia. Online continual learning with maximal interfered retrieval. *Advances in neural information processing systems*, 32, 2019.

[3] Rahaf Aljundi, Min Lin, Baptiste Goujaud, and Yoshua Bengio. Gradient based sample selection for online continual learning. *Advances in neural information processing systems*, 32, 2019.

[4] Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark experience for general continual learning: a strong, simple baseline. *Advances in neural information processing systems*, 33:15920–15930, 2020.

[5] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021.

[6] Francisco M Castro, Manuel J Marín-Jiménez, Nicolás Guil, Cordelia Schmid, and Karteek Alahari. End-to-end incremental learning. In *Proceedings of the European conference on computer vision (ECCV)*, pages 233–248, 2018.

[7] Arslan Chaudhry, Marc'Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with a-gem. *arXiv preprint arXiv:1812.00420*, 2018.

[8] Xiuwei Chen and Xiaobin Chang. Dynamic residual classifier for class incremental learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 18743–18752, 2023.

[9] Aristotelis Chrysakis and Marie-Francine Moens. Online bias correction for task-free continual learning. *ICLR 2023 at OpenReview*, 2023.

[10] Matthias De Lange and Tinne Tuytelaars. Continual prototype evolution: Learning online from non-stationary data streams. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 8250–8259, 2021.

[11] Prithviraj Dhar, Rajat Vikram Singh, Kuan-Chuan Peng, Ziyan Wu, and Rama Chellappa. Learning without memorizing. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5138–5146, 2019.

[12] Songlin Dong, Yingjie Chen, Yuhang He, Yuhan Jin, Alex C Kot, and Yihong Gong. Analogical augmentation and significance analysis for online task-free continual learning. *IEEE Transactions on Multimedia*, 27:3370–3382, 2025.

[13] Arthur Douillard, Alexandre Ramé, Guillaume Couairon, and Matthieu Cord. Dytox: Transformers for continual learning with dynamic token expansion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9285–9295, 2022.

[14] Zhanxin Gao, Jun Cen, and Xiaobin Chang. Consistent prompting for rehearsal-free continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 28463–28473, 2024.

[15] Yiduo Guo, Bing Liu, and Dongyan Zhao. Dealing with cross-task class discrimination in online continual learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11878–11887, 2023.

[16] Jiangpeng He and Fengqing Zhu. Online continual learning via candidates voting. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 3154–3163, 2022.

[17] Jiangpeng He, Zhihao Duan, and Fengqing Zhu. Cl-lora: Continual low-rank adaptation for rehearsal-free class-incremental learning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 30534–30544, 2025.

[18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[19] Yuhang He, Yingjie Chen, Yuhan Jin, Songlin Dong, Xing Wei, and Yihong Gong. Dyson: Dynamic feature space self-organization for online task-free class incremental learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 23741–23751, 2024.

[20] Dan Hendrycks, Steven Basart, Norman Mu, Saurav Kadavath, Frank Wang, Evan Dorundo, Rahul Desai, Tyler Zhu, Samyak Parajuli, Mike Guo, Dawn Song, Jacob Steinhardt, and Justin Gilmer. The many faces of robustness: A critical analysis of out-of-distribution generalization. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 8340–8349, 2021.

[21] Yongwon Hong, Sungho Park, and Hyeran Byun. Task-aware network: Mitigation of task-aware and task-free performance gap in online continual learning. *Neurocomputing*, 552:126527, 2023.

[22] Fushuo Huo, Wenchao Xu, Jingcai Guo, Haozhao Wang, and Yunfeng Fan. Non-exemplar online class-incremental continual learning via dual-prototype self-augment and refinement. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 12698–12707, 2024.

[23] Xisen Jin, Arka Sadhu, Junyi Du, and Xiang Ren. Gradient-based editing of memory examples for online task-free continual learning. *Advances in Neural Information Processing Systems*, 34:29193–29205, 2021.

[24] Ronald Kemker and Christopher Kanan. Fearnet: Brain-inspired model for incremental learning. *arXiv preprint arXiv:1711.10563*, 2017.

[25] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.

[26] Hyunseo Koh, Minhyuk Seo, Jihwan Bang, Hwanjun Song, Deokki Hong, Seulki Park, Jung-Woo Ha, and Jonghyun Choi. Online boundary-free continual learning by scheduled data prior. In *The Eleventh International Conference on Learning Representations*, 2023.

[27] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

[28] Soochan Lee, Junsoo Ha, Dongsu Zhang, and Gunhee Kim. A neural dirichlet process mixture model for task-free continual learning. *arXiv preprint arXiv:2001.00689*, 2020.

[29] Jin Li, Zhong Ji, Gang Wang, Qiang Wang, and Feng Gao. Learning from students: Online contrastive distillation network for general continual learning. In *IJCAI*, pages 3215–3221, 2022.

[30] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2935–2947, 2017.

[31] Huiwei Lin, Baoquan Zhang, Shanshan Feng, Xutao Li, and Yunming Ye. Pcr: Proxy-based contrastive replay for online class-incremental continual learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 24246–24255, 2023.

[32] Xialei Liu, Marc Masana, Luis Herranz, Joost Van de Weijer, Antonio M Lopez, and Andrew D Bagdanov. Rotate your networks: Better weight consolidation and less catastrophic forgetting. *arXiv preprint arXiv:1802.02950*, 2018.

[33] Vincenzo Lomonaco and Davide Maltoni. Core50: a new dataset and benchmark for continuous object recognition. In *Conference on robot learning*, pages 17–26. PMLR, 2017.

[34] Arun Mallya and Svetlana Lazebnik. Packnet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 7765–7773, 2018.

[35] Saleh Momeni, Sahisnu Mazumder, and Bing Liu. Continual learning using a kernel-based method over foundation models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 19528–19536, 2025.

[36] Saleh Momeni, Changnan Xiao, and Bing Liu. Continual out-of-distribution detection with analytic neural collapse. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 15564–15572, 2026.

[37] Jun-Yeong Moon, Keon-Hee Park, Jung Uk Kim, and Gyeong-Moon Park. Online class incremental learning on stochastic blurry task boundary via mask and visual prompt tuning. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 11731–11741, 2023.

[38] Parinita Nema and Vinod K Kurmi. Strategic base representation learning via feature augmentations for few-shot class incremental learning. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 6394–6403. IEEE, 2025.

[39] Julien Pourcel, Ngoc-Son Vu, and Robert M French. Online task-free continual learning with dynamic sparse distributed memory. In *European Conference on Computer Vision*, pages 739–756. Springer, 2022.

[40] Ameya Prabhu, Philip HS Torr, and Puneet K Dokania. Gdumb: A simple approach that questions our progress in continual learning. In *European conference on computer vision*, pages 524–540. Springer, 2020.

[41] Julien Rabin, Gabriel Peyré, Julie Delon, and Marc Bernot. Wasserstein barycenter and its application to texture mixing. In *International conference on scale space and variational methods in computer vision*, pages 435–446. Springer, 2011.

[42] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 2001–2010, 2017.

[43] Deboleena Roy, Priyadarshini Panda, and Kaushik Roy. Tree-cnn: a hierarchical deep convolutional neural network for incremental learning. *Neural networks*, 121:148–160, 2020.

[44] Andrei A Rusu, Neil C Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.

[45] AK Md Ehsanes Saleh, Mohammad Arashi, and BM Golam Kibria. *Theory of ridge regression estimation with applications*. John Wiley & Sons, 2019.

[46] Joan Serra, Didac Suris, Marius Miron, and Alexandros Karatzoglou. Overcoming catastrophic forgetting with hard attention to the task. In *International conference on machine learning*, pages 4548–4557. PMLR, 2018.

[47] Murray Shanahan, Christos Kaplanis, and Jovana Mitrović. Encoders and ensembles for task-free continual learning. *arXiv preprint arXiv:2105.13327*, 2021.

[48] Dongsub Shim, Zheda Mai, Jihwan Jeong, Scott Sanner, Hyunwoo Kim, and Jongseong Jang. Online class-incremental continual learning with adversarial shapley value. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 9630–9638, 2021.

[49] James Seale Smith, Leonid Karlinsky, Vyshnavi Gutta, Paola Cascante-Bonilla, Donghyun Kim, Assaf Arbelle, Rameswar Panda, Rogerio Feris, and Zsolt Kira. Coda-prompt: Continual decomposed attention-based prompting for rehearsal-free continual learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11909–11919, 2023.

[50] Xiaoyu Tao, Xinyuan Chang, Xiaopeng Hong, Xing Wei, and Yihong Gong. Topology-preserving class-incremental learning. In *European conference on computer vision*, pages 254–270. Springer, 2020.

[51] Aopeng Wang, Ke Deng, Yongli Ren, and Jun Luo. Rehearsal-free and task-free online continual learning with contrastive prompt. *arXiv preprint arXiv:2510.00467*, 2025.

[52] Fu-Yun Wang, Da-Wei Zhou, Han-Jia Ye, and De-Chuan Zhan. Foster: Feature boosting and compression for class-incremental learning. In *European conference on computer vision*, pages 398–414. Springer, 2022.

[53] Kai Wang, Joost van de Weijer, and Luis Herranz. Acae-remind for online continual learning with compressed feature replay. *Pattern Recognition Letters*, 150:122–129, 2021.

[54] Shaokun Wang, Weiwei Shi, Songlin Dong, Xinyuan Gao, Xiang Song, and Yihong Gong. Semantic knowledge guided class-incremental learning. *IEEE Transactions on Circuits and Systems for Video Technology*, 33(10):5921–5931, 2023.

[55] Zhen Wang, Liu Liu, Yajing Kong, Jiaxian Guo, and Dacheng Tao. Online continual learning with contrastive vision transformer. In *European Conference on Computer Vision*, pages 631–650. Springer, 2022.

[56] Zhenyi Wang, Li Shen, Le Fang, Qiuling Suo, Tiehang Duan, and Mingchen Gao. Improving task-free continual learning by distributionally robust memory evolution. In *International conference on machine learning*, pages 22985–22998. PMLR, 2022.

[57] Zifeng Wang, Zizhao Zhang, Sayna Ebrahimi, Ruoxi Sun, Han Zhang, Chen-Yu Lee, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, et al. Dualprompt: Complementary prompting for rehearsal-free continual learning. In *European conference on computer vision*, pages 631–648. Springer, 2022.

[58] Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, and Tomas Pfister. Learning to prompt for continual learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 139–149, 2022.

[59] Xiwen Wei, Guihong Li, and Radu Marculescu. Online-lora: Task-free online continual learning via low rank adaptation. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 6634–6645. IEEE, 2025.

[60] Yujie Wei, Jiaxin Ye, Zhizhong Huang, Junping Zhang, and Hongming Shan. Online prototype learning for online continual learning. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 18764–18774, 2023.

[61] Peter Welinder, Steve Branson, Takeshi Mita, Catherine Wah, Florian Schroff, Serge Belongie, and Pietro Perona. Caltech-ucsd birds 200. 2010.

[62] Ye Xiang, Ying Fu, Pan Ji, and Hua Huang. Incremental learning using conditional adversarial networks. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 6619–6628, 2019.

[63] Shipeng Yan, Jiangwei Xie, and Xuming He. Der: Dynamically expandable representation for class incremental learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 3014–3023, 2021.

[64] Yibo Yang, Haobo Yuan, Xiangtai Li, Zhouchen Lin, Philip Torr, and Dacheng Tao. Neural collapse inspired feature-classifier alignment for few-shot class incremental learning. *arXiv preprint arXiv:2302.03004*, 2023.

[65] Fei Ye and Adrian G Bors. Task-free continual learning via online discrepancy distance learning. *Advances in Neural Information Processing Systems*, 35:23675–23688, 2022.

[66] Fei Ye and Adrian G Bors. Learning dynamic latent spaces for lifelong generative modelling. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 10891–10899, 2023.

[67] Fei Ye and Adrian G Bors. Online task-free continual learning via dynamic expansionable memory distribution. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 20512–20522, 2025.

[68] Felix Xinnan X Yu, Ananda Theertha Suresh, Krzysztof M Choromanski, Daniel N Holtmann-Rice, and Sanjiv Kumar. Orthogonal random features. *Advances in neural information processing systems*, 29, 2016.

[69] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *International conference on machine learning*, pages 3987–3995. Pmlr, 2017.

[70] Huiping Zhuang, Yizhu Chen, Di Fang, Run He, Kai Tong, Hongxin Wei, Ziqian Zeng, and Cen Chen. Gacl: Exemplar-free generalized analytic continual learning. *Advances in neural information processing systems*, 37:83024–83047, 2024.